

IOAI lauks

- **Laika ierobežojums:** 5 minūtes
- **Krātuve:** 5 GB
- **Risinājuma izmērs:** `solution.ipynb`, `custom_model.py` kopā ≤ 1 MB
- **Iepriekš apmācīti modeļi:** nav — apmāciet no nulles, vērtēšanas laikā nav interneta
- **Baseline rezultāts:** 31.2187

Uzdevums

Astana mērs vēlas izrotāt pilsētu ar stilizētiem IOAI logotipiem. Kā statistiķis viņš visu — tostarp arī logotipu — uzskata par telpisku funkciju $F(x, y, \overline{W})$, kur $x, y \in [0, 1]$ apzīmē koordinātas 2D plaknē un $\overline{W}$ ir slēpto parametru kopa, kas definē stilistiskās īpašības, tādas kā burtu krāsas un leņķus.

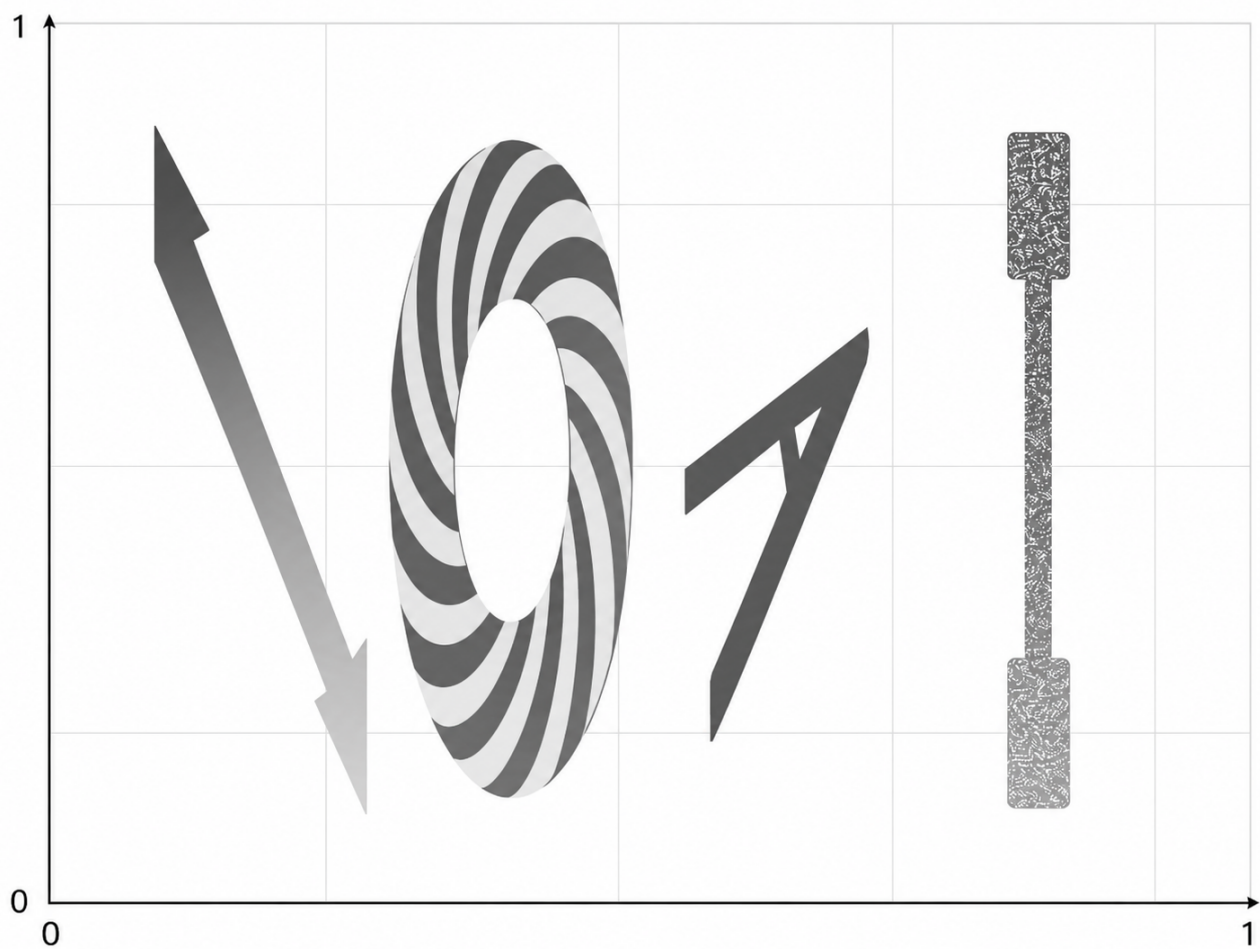
Tā kā F ir pārāk sarežģīta, lai to izteiktu ar skaidru matemātisku vienādojumu, jūsu uzdevums ir apmācīt neironu tīklu, kas to aptuveni atveido. Tīkls izvadīs **IOAI lauka** vērtību jebkuram koordinātu pārim (x, y) , ģenerējot pilnu logotipa siltumkartes (heatmap) vizualizāciju visā plaknē. Šeit ir piemērs F siltumkartes vizualizācijai ar dažiem konkrētiem slēptajiem parametriem $\overline{W}$.

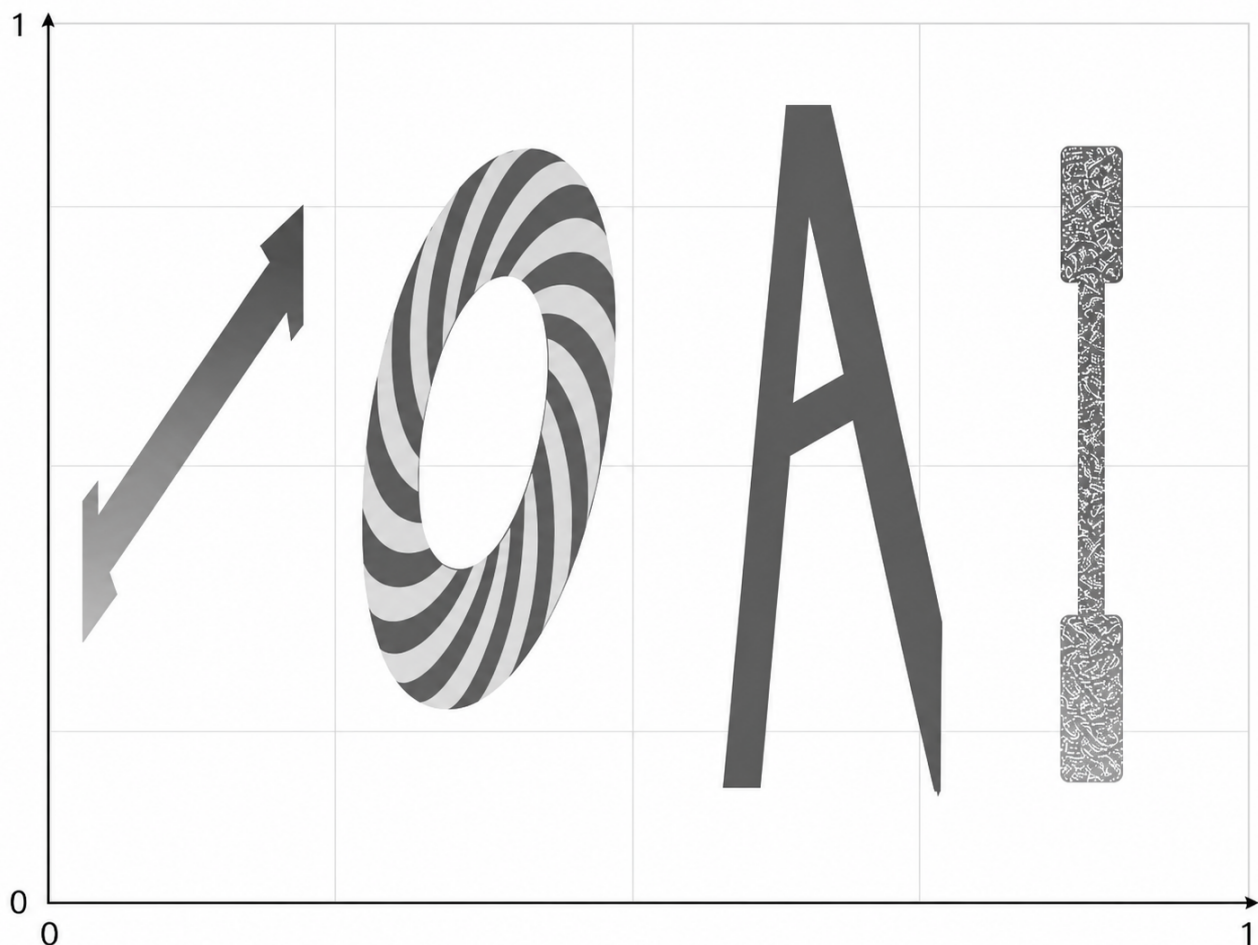


No kā sastāv IOAI lauks? No četriem burtiem un fona.

- Vērtības pirmā burtā I iekšpusē ir ļoti lielas ($1e+10$ un vairāk) ar lineāru gradientu
- Vērtības burtā O veido spirālveida rakstu
- Vērtība burtā A iekšpusē vienmēr ir -1
- Vērtībām pēdējā burtā I iekšpusē jābūt nejaušām vērtībām no intervāla $[-2026, 2026]$ pat tad, ja tās tiek aprēķinātas divas reizes vienā un tajā pašā punktā
- Ārpus burtiem vērtība vienmēr ir nulle

Funkcijai ir slēptie parametri $\overline{W}$, kas ietekmē burtu mērogu un slīpumu, kā arī vērtību diapazonu pirmā burtā I iekšpusē. Tomēr burti nekrustosies. Šeit ir daži ilustratīvi piemēri, kā IOAI lauks izskatās ar dažādiem $\overline{W}$:





Kas jums ir dots:

Šajā uzdevumā NAV datu kopu. Tā vietā jums ir dota ģeneratora funkcija, ko konfigurē JSON konfigurācijas fails `data/train_config/field_config.json`.

Testa konfigurācija ir slēpta, taču tā ir līdzīgas dabas. Jūsu uzdevums ir pielāgoties dotajam ģeneratoram, izmantojot tik daudz datu, cik vēlaties. Jūsu "train" un "test" sadalījumi tiek ģenerēti no viena un tā paša ģeneratora — jūs vienkārši nezināt, kuros punktos (x_i, y_i) jūs tiksiet novērtēti.

Jūsu iesūtījumam jāsatāv no:

- apmācāmā modeļa klases, kas saglabāta kā `custom_model.py`. Šim modelim jāņem no `torch.nn.Module` klases un jāizmanto tikai `torch` importi. Tam jāsaturs `CustomModel` klase, kas tiek izmantota `solution.ipynb` piezīmju grāmatā.
- `solution.ipynb` piezīmju grāmatas, kas izveidos `model.pt` svarus

Vērtēšana

Katram apgabalam minimālais rezultāts ir 0 un maksimālais rezultāts ir 1. Galīgais rezultāts tiek vidējots pār visiem pieciem apgabaliem (četriem — pa vienam katram burtam — un fonu) un pareizināts ar 100. Ir **parametru sods**:

Ja jūsu modelim ir vairāk nekā 20260 parametru, rezultāts tiek samazināts uz pusi.

Parametru skaits tiek mērīts ar `sum(p.numel() for p in model.parameters())`. Mēs sagaidām, ka jūsu modelis darbosies arī stohastiskā režīmā, PyTorch `nn.Dropout` būdams modeļa daļa.

Standarta apgabaliem

Katram apgabalam R (pirmais burts `I`, `O`, `A`, `Background`) mēs novērtējam modeli $N_R = 512$ testa punktos (x_i, y_i) ar patiesajām vērtībām v_i un prognozēm $\hat{v}_i$. Kā galveno metriku mēs izmantojam normalizētu vidējo absolūto kļūdu (Mean Absolute Error, MAE). MAE tiek definēta kā:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Un normalizācija tiek veikta šādi

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

kur $s_R > 0$ ir mēroga konstante.

Pēdējā burta `I` apgabalam

Šajā apgabalā **dropout ir ieslēgts novērtēšanas laikā**. Katram testa punktam j :

1. Mēs palaižam modeli $K = 10$ reizes, lai iegūtu $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Ja kāda izvade ir ārpus intervāla $[-2026, 2026]$, tad `pointScore(j) = 0`.
3. Pretējā gadījumā aprēķinām K izvažu standartnovirzi σ_j un pārvēršam to rezultātā:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

kur $s_E > 0$ ir fiksēta mēroga konstante.

Apgabala rezultāts ir vidējais pār visiem apgabala punktiem:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

kur $N_E = K * N_R$.

Vienkāršiem vārdiem — jo lielāka daudzveidība jums ir, jo lielāks būs jūsu rezultāts šajā apgabalā. **Jūs nevarat izmantot nejaušību tiešā veidā, tostarp PyTorch `rand*` un `_uniform` funkcijas; nejaušībai jārodas no inferences ar ieslēgtu dropout.**

Kā iesūtīt

1. Atveriet `solution.ipynb` un izpildiet visas šūnas.
2. Uzlabojiet `CustomModel` modeli failā `custom_model.py`
3. Pārliecinieties, ka jūsu pēdējā šūna saglabā jūsu modeli `model.pt` failā.
4. JupyterLab Git cilnē veiciet `solution.ipynb` un `custom_model.py` stage, komentējiet un commit, tad veiciet push.
5. Atgriezieties Contest lapā un noklikšķiniet **Submit**. Iesūtīšanas komentāram jābūt tādām pašām kā komentāram no iepriekšējā soļa.